Demo_0

October 28, 2026

[1]:
```
print("Hello World")
```

```
Hello World
```

[ ]:
```
# This is a comment
```

[4]:
```
# Set a variable

message = "Hello World"
```

[5]:
```
print(message)
```

```
Hello World
```

[6]:
```
# Data Types

"Hello World" # this is a string

1 # This is an integer

3.14159 # This is a Float

True # Boolean
```

[6]:
```
True
```

[9]:
```
# Data Structures

cities = ["Minneapolis", "Chicago", "Milwaukee"] # A List

coords = (43, 94) # A Tuple

temp_dict = {"Minneapolis":70, "Chicago":72} # A dictionary
```

[11]:
```
# Accessing Elements of Data Structures

print(cities[0]) # Print the first element of list (Python starts counting at 0)
```

```
Minneapolis
```

[13]:
```
print(temp_dict["Minneapolis"]) # Get value assigned to 'Minneapolis' from␣
↪dictionary
```

```
70
```

[20]:
```
# Operations

1 + 1 # Addition
1 - 1 # Subtraction
1 * 9 # Multiplication
1/9 # Division

'Hello' + 'World' # String Concatenation

# Booleans
is_raining = True
is_sunny = False

is_raining and is_sunny # AND
is_raining or is_sunny # OR
```

[20]:
```
True
```

[26]:
```
# Conditions

1 == 2 # Is equal to

1 != 2 # Is not equal to

1 > 0 # Greater Than

1 < 0 # Less Than
```

[26]:
```
False
```

[32]:
```
# If Statements INDENTATION is important!!!

is_raining = False

if is_raining:
    print("it is raining")
elif is_sunny:
    print("it is sunny")
else:
    print('it is not sunny nor rainy')
```

```
print('Hello World')
```

```
it is not sunny nor rainy
Hello World
```

[34]:
```
# For Loops (AKA iteration)

for city in cities: # For each city in this list of cities
    # Do these things
    print('Hello')
    print(city)

    print('\n')
```

```
Hello
Minneapolis


Hello
Chicago


Hello
Milwaukee
```

[35]:
```
# Functions

help(print) # Help is a function that pulls up documentation on another function
```

```
Help on built-in function print in module builtins:

print(*args, sep=' ', end='\n', file=None, flush=False)
    Prints the values to a stream, or to sys.stdout by default.

    sep
      string inserted between values, default a space.
    end
      string appended after the last value, default a newline.
    file
      a file-like object (stream); defaults to the current sys.stdout.
    flush
      whether to forcibly flush the stream.
```

[38]:
```
for number in range(9):
    print(number)
```

```
0
1
2
3
4
5
6
7
8
```